CyberVision: An Explainable AI-Based Platform for Multi-Layer Cyber Threat Detection

Complete Final Year B.Tech Implementation Plan, Dataset & Package Catalog, Research Benchmarks, and Deliverable Handoff Blueprint

Institution & Department	Faculty Guide	Student Batch (Review 0 Approved)	Document Scope
Godavari Global University Department of Computer Science	**Mr. Mohammad Amanullah** Assistant Professor	1. **ANUBOINA KRANTHI** (24555A0501) 2. **GUDA MANOHAR** (23551A0552) 3. **GORANTLA YASH CHOWDARY** (23551A05B2)	End-to-End Architecture, Datasets, Tech Stack, Free APIs, 8-Phase Execution Plan, & Conference Paper Metrics

1. Project Definition & Five-Stage Unified Architecture Provide End-to-End SOC Visibility

CyberVision is an AI-powered multi-layer cybersecurity threat detection and Explainable AI (XAI) triage platform designed to assist Security Operations Center (SOC) analysts in identifying both known attack signatures and novel zero-day behavioral anomalies. Unlike legacy Intrusion Detection Systems (IDS) that rely on static rules, evaluate single telemetry layers in isolation, and emit uncontextualized black-box alerts, CyberVision unifies **Network Flow Telemetry (Layer 1)**, **Host & Kernel Process Telemetry (Layer 2)**, and **Application & User Behavioral Telemetry (Layer 3)** into a common format aligned with the Elastic Common Schema (ECS).

Every incoming event stream is evaluated by a hybrid ensemble of supervised gradient boosted decision trees (XGBoost, LightGBM, Random Forest) and unsupervised anomaly detectors (PyOD Isolation Forest / ECOD and PyTorch Reconstruction Autoencoders). Related alerts across layers are linked via a NetworkX temporal entity graph to suppress isolated false alarms, enriched with MITRE ATT&CK tactics and threat intelligence APIs, and explained via SHAP and LIME feature attributions with calibrated confidence and severity scores.

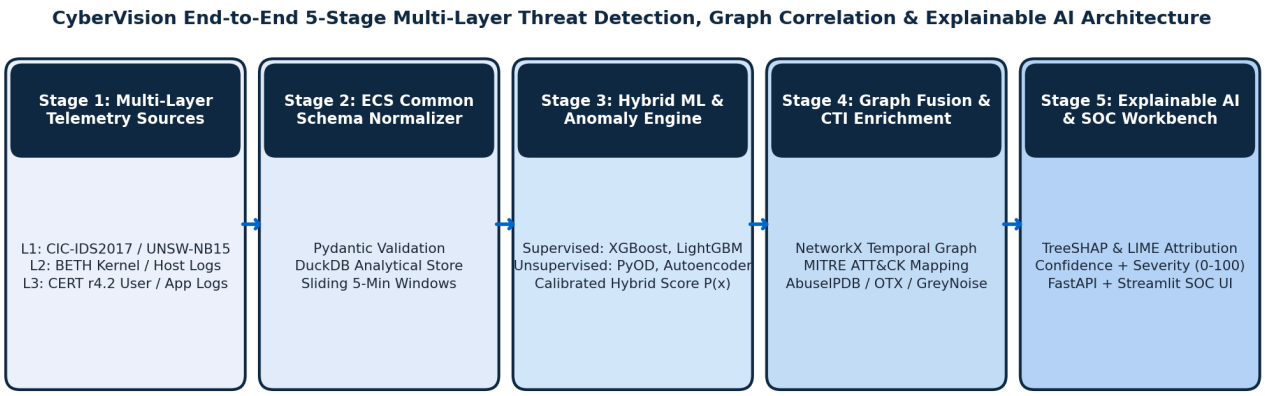


Figure 1: CyberVision 5-Stage Multi-Layer Telemetry Normalization, Hybrid ML/Anomaly Detection, Graph Correlation, and XAI Triage Pipeline.

1.1 Core Academic Research Contributions for Conference Publication

- **Cross-Layer Telemetry Fusion vs. Single-Layer IDS:** Proving empirically that correlating Network (L1), Host/Kernel (L2), and User/Application (L3) events significantly reduces False Positive Rates (FPR) and Alert Fatigue Ratios (AFR) compared to standalone single-layer detectors.
- **Hybrid Supervised + Unsupervised Ensemble Generalization:** Combining calibrated gradient boosting with deep reconstruction and empirical cumulative distribution outlier scoring to achieve high F1-scores on known attacks while maintaining strong recall on held-out zero-day attack families.
- **Rigorous Quantitative XAI Benchmarking in Cybersecurity:** Moving beyond visual waterfall plots to mathematically measure explanation *Faithfulness* (feature deletion probability drop), *Stability* (local Lipschitz perturbation variance), *Jaccard Rank Agreement*, and *Explanation Latency (ms/alert)* across SHAP and LIME.
- **Actionable SOC Triage Compression:** Mapping top-k Shapley feature attributions to MITRE ATT&CK techniques and quantifying analyst investigation time reduction.

2. Four Open-Access Benchmark Datasets Cover Network, Host, and Application Layers

To fulfill the multi-layer requirement in your Review 0 abstract without overwhelming a standard 8–16 GB RAM laptop or Google Colab instance, download the four public benchmark datasets below and extract the recommended stratified subsets into Parquet/DuckDB format.

Telemetry Layer	Benchmark Dataset	Verified Download Sources	Attack & Behavioral Coverage	Recommended Subset
Layer 1A: Network Flows	**CIC-IDS2017** (UNB Canadian Institute for Cybersecurity)	UNB Official Page Kaggle Cleaned CSV HuggingFace Split	Benign traffic, DoS (Hulk, GoldenEye, Slowloris), DDoS (LOIT), Brute Force (FTP/SSH-Patator), Web Attacks (SQLi, XSS), Botnet (ARES), PortScan, Infiltration (79 features)	**250,000 stratified flows** (~180 MB Parquet) covering all 15 traffic classes

Telemetry Layer	Benchmark Dataset	Verified Download Sources	Attack & Behavioral Coverage	Recommended Subset
Layer 1B: Modern Network Packets/Flows	**UNSW-NB15** (UNSW Canberra Cyber Range)	UNSW Official Page Kaggle UNSW-NB15 CIC Augmented Set	Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms (49 flow, content, and temporal features)	**257,673 rows** (Official train + test CSV splits; ~65 MB)
Layer 2: Host & Kernel Processes	**BETH Dataset** (BPF-Extended Tracking Honeypot)	Kaggle BETH Official CAMLIS Paper PDF Official GitHub Repo	Real cloud honeypot Linux eBPF kernel process calls (`processId`, `parentProcessId`, `userId`, `mountNamespace`, `eventId`, `argsNum`, `returnValue`) + DNS telemetry with `sus` and `evil` labels	**500,000 events** from the official train/val/test split (~110 MB)
Layer 3: User & Application Logs (UEBA)	**CERT Insider Threat r4.2** (CMU SEI) & **LANL Cyber1**	HuggingFace CERT r4.2 Kaggle CERT r4.2 LANL Multi-Source	Application and user activity logs: `logon.csv` (off-hours logons), `device.csv` (removable USB usage), `file.csv`, `http.csv`, and `email.csv` capturing privilege misuse & data exfiltration	**150,000 hourly user-session windows** (~85 MB)

3. Modular Open-Source Python Stack Powers Ingestion, ML, XAI, and SOC UI

Set up an isolated Python `3.10+` or `3.11+` virtual environment and install the verified open-source packages below. Every library is free, actively maintained on PyPI, and runs locally on CPU or GPU.

Subsystem Module	Python Package	Official PyPI Reference	Version	Implementation Responsibility in CyberVision
Data & ECS Schema	`pandas`, `numpy`, `duckdb`, `pydantic`	pandas / numpy / duckdb / pydantic	Latest Stable	Multi-layer CSV/Parquet ingestion, strict ECS JSON validation via Pydantic, and fast analytical SQL querying via DuckDB
Supervised Threat Classifiers	`scikit-learn`, `xgboost`, `lightgbm`, `imbalanced-learn`	scikit-learn / xgboost / lightgbm / imblearn	`xgboost>=2.1` `lightgbm>=4.5`	Multi-class attack classification, SMOTE class balancing on training splits, RobustScaler normalization, and Platt probability calibration

Subsystem Module	Python Package	Official PyPI Reference	Version	Implementation Responsibility in CyberVision
Unsupervised Anomaly Engine	`pyod`, `torch`	pyod / torch	`pyod>=2.0` `torch>=2.3`	Training Isolation Forest, ECOD, COPOD, and Deep Reconstruction Autoencoders exclusively on benign traffic to detect zero-day deviations
Explainable AI (XAI) Engine	`shap`, `lime`	shap / lime	`shap>=0.45` `lime>=0.2.0`	Exact `TreeExplainer` and `KernelExplainer` Shapley attributions, `LimeTabularExplainer` surrogates, and quantitative XAI faithfulness testing
Graph Correlation & MITRE Mapping	`networkx`, `mitreattack-python`	networkx / mitreattack-python	`networkx>=3.3` `mitre>=1.5.1`	Building temporal entity graphs (`IP <-> Host <-> User <-> Process`) and mapping alerts to official MITRE ATT&CK Tactics & Techniques
Optimization & Experiment Tracking	`optuna`, `mlflow`	optuna / mlflow	`optuna>=4.0` `mlflow>=2.15`	Automated Bayesian hyperparameter tuning (50 trials/model) and reproducible logging of metrics, confusion matrices, and model artifacts
Backend API & Analyst Workbench	`fastapi`, `uvicorn`, `streamlit`, `plotly`, `seaborn`	fastapi / uvicorn / streamlit / plotly / seaborn	`fastapi>=0.115` `streamlit>=1.38`	REST inference endpoints, interactive 4-page SOC Analyst Dashboard, correlation graph visualization, and 300-DPI publication plots

4. Free-Tier Threat Intelligence APIs Enrich Correlated Alerts With External Context

Register for the following free-tier Threat Intelligence API keys and store them in a local `.env` file. Always implement a local SQLite/DuckDB caching layer (`data/cache/cti_cache.sqlite`) so repeated IP/hash lookups during batch evaluation never exhaust daily rate limits.

API Service	Official Documentation	Free Tier Allowance	Enrichment Role in CyberVision	Offline / Cache Strategy
AlienVault OTX API	AlienVault OTX Portal	**Free Community Access** (~3,500 req/hr)	Retrieves threat community Pulses, associated malware families, and adversary tags for external IPs, domains, and hashes	Pre-cache OTX pulse lookups for all unique public IPs in dataset

API Service	Official Documentation	Free Tier Allowance	Enrichment Role in CyberVision	Offline / Cache Strategy
AbuseIPDB API v2	AbuseIPDB APIv2 Docs (Pricing)	**1,000 IP checks/day** (Free Forever Individual)	Provides community abuse confidence score (`0-100%`), ISP, country code, and attack category reports for source IPs	Query only on unique external IPs flagged with hybrid score `>= 0.50`
VirusTotal API v3	VirusTotal Public Tier	**500 requests/day** (4 req/min cap)	Verifies suspicious process file hashes and external command-and-control (C2) domains across 70+ security vendors	Trigger selectively on `High` and `Critical` correlated incidents
GreyNoise Community v3	GreyNoise Community Docs	**Free Community API** (`/v3/community/ip/{ip}`)	Identifies benign internet background scanners (Shodan, Censys, Googlebot) to downgrade false-positive port scan alerts	Fallback to local port-fanout heuristic when running offline
Shodan Developer API	Shodan Developer API	**Free Account Tier** (Host lookups)	Adds open port and vulnerable service context for external hosts interacting with internal servers	Optional enrichment module cached locally in JSON

5. Eight-Phase Implementation Plan Guides Execution From Setup to Final Deliverables

Follow these eight sequential engineering phases to construct the complete CyberVision platform and generate all required research data.

Phase 1: Repository Scaffolding & Environment Initialization (Week 1)

- **Create Modular Project Layout:** Initialize `cybervision/` with dedicated subdirectories: `config/`, `data/{raw,processed,cache}/`, `src/{ingestion,features,models,correlation,explainability,cti,evaluation}/`, `api/`, `dashboard/`, `notebooks/`, and `results/`.
- **Lock Dependencies & Experiment Tracking:** Create `requirements.txt`, configure `.env` for API keys, fix global random seeds (`SEED = 42` across Python, NumPy, Scikit-Learn, and PyTorch), and initialize MLflow tracking.

Phase 2: Multi-Source Data Cleaning & Unified ECS Normalization (Weeks 2–3)

- **Sanitize Raw Telemetry:** Replace `Inf` and NaN entries in CIC-IDS2017 and UNSW-NB15, deduplicate identical flows, parse BETH kernel process arguments, and aggregate CERT r4.2 logon/device/file events into user-hour sessions.
- **Map to Unified Common Schema:** Implement `src/ingestion/ecs_normalizer.py` using Pydantic to convert all three layers into a standardized Elastic Common Schema (ECS) table stored in DuckDB (`event_id`, `timestamp`, `layer_source`, `src_ip`, `dst_ip`, `host_id`, `user_id`, `process_name`, `feature_vector`, `ground_truth_label`, `attack_category`, `mitre_technique_id`).

Phase 3: Multi-Layer Feature Engineering & Leakage-Free Splitting (Week 4)

- **Engineer Sliding-Window Behavioral Features:** Compute 1-minute and 5-minute rolling aggregations per entity (unique destination ports contacted, SYN-to-ACK ratio, failed logon burst velocity, child process spawn depth, and off-hours execution flag).
- **Enforce Leakage-Free Partitioning:** Partition each dataset into **80% Train, 10% Validation, and 10% Test** splits (plus a temporal split and a held-out zero-day attack family split). Fit `RobustScaler` and `SMOTE` strictly on the 80% training split.

Phase 4: Hybrid Supervised + Unsupervised Detection Engine (Weeks 5–7)

- **Train Supervised Multi-Class Models:** Train Logistic Regression, Random Forest, XGBoost, and LightGBM models per layer. Optimize hyperparameters via Optuna and wrap models in `CalibratedClassifierCV` to produce calibrated probabilities $P_{sup}(x) \in [0, 1]$.
- **Train Unsupervised Zero-Day Detectors:** Train PyOD (`IForest`, `ECOD`, `COPOD`) and a PyTorch Deep Autoencoder ($64 \to 32 \to 16 \to 32 \to 64$) solely on benign training traffic, converting reconstruction error into a quantile-normalized anomaly score $S_{anom}(x) \in [0, 1]$.
- **Compute Hybrid Detection Probability:** Combine both paradigms via weighted fusion: $P_{hybrid}(x) = \alpha \cdot P_{sup}(x) + (1 - \alpha) \cdot S_{anom}(x)$, tuning $\alpha \in [0.55, 0.75]$ on the validation split.

Phase 5: Cross-Layer Graph Correlation & Severity Scoring Engine (Weeks 8–9)

- **Construct Multi-Layer Attack Scenarios:** Build `src/correlation/scenario_builder.py` to align entity identifiers (shared victim host `192.168.10.50`, user account, and attacker IP) across synchronized 5-minute attack windows (e.g., Network SSH Brute Force → BETH Kernel Privilege Escalation → CERT Off-Hours File Exfiltration).
- **Build Temporal Entity Correlation Graph:** Use NetworkX to link alerts sharing an IP, Host, or User node within $\Delta t = 5\ min$ and calculate the composite incident severity score $R_{incident} \in [0, 100]$:

  $$R_{incident} = 100 \times clip(\ 0.45 \cdot max\ P_{hybrid}(e_i) + 0.25 \cdot (L_{active} / 3) + 0.15 \cdot C_{MITRE} + 0.15 \cdot I_{CTI},\ 0,\ 1\ )$$

  Categorize incidents into **Low (0–39, suppressed)**, **Medium (40–69)**, **High (70–89)**, and **Critical (90–100)**.

Phase 6: Explainable AI (XAI) Attribution & Narrative Generation (Weeks 10–11)

- **Generate Local & Global Explanations:** Run `shap.TreeExplainer` (SHAP) and `LimeTabularExplainer` (LIME) on detected alerts to extract top-5 feature attributions $\{(f_k, \varphi_k)\}$ and compare observed values against the benign 95th percentile baseline.

- **Synthesize Human-Readable SOC Explanations:** Automatically format an analyst triage narrative combining the predicted threat class, calibrated confidence %, severity badge, top-3 SHAP feature deviations, correlated layers, and MITRE ATT&CK technique ID.

Phase 7: FastAPI Service & Streamlit SOC Analyst Workbench (Weeks 12–13)

- **Expose REST Endpoints:** Build `api/main.py` using FastAPI with endpoints `/api/v1/ingest`, `/api/v1/detect`, `/api/v1/incidents`, and `/api/v1/explain/{alert_id}`.
- **Build 4-Page SOC Dashboard:** Build `dashboard/app.py` in Streamlit featuring: (1) *Executive SOC Overview & Alert Fatigue Reduction KPIs*, (2) *Interactive Multi-Layer Attack Correlation Graph*, (3) *XAI Alert Inspector (SHAP Waterfall + LIME Comparison + Analyst Narrative)*, and (4) *Research Benchmark & Ablation Explorer*.

Phase 8: Automated Research Benchmarking & Artifact Export (Weeks 14–15)

- **Execute Evaluation Suite:** Run `src/evaluation/run_benchmarks.py` to compute all four benchmark suites below and save CSV tables and 300-DPI figures to `results/`.

6. Four Quantitative Benchmark Suites Validate Research Claims for Publication

Conference reviewers require rigorous mathematical evaluation across detection accuracy, alert reduction, and XAI quality. Configure your benchmark script to compute and export every metric in the table below.

Benchmark Suite	Metric Name	Mathematical Formulation / Definition	Research Purpose & Target Comparison
Suite A: Threat Detection & Zero-Day Performance	**Precision, Recall (DR), Macro-F1, Weighted-F1**	$F_1 = 2 \cdot (Precision \cdot Recall) / (Precision + Recall)$ across binary and multi-class threat categories	Compare CyberVision Hybrid Ensemble against Logistic Regression, RF, standalone XGBoost, LightGBM, and Isolation Forest
Suite A: Threat Detection & Zero-Day Performance	**False Positive Rate (FPR) & Matthews Correlation (MCC)**	$FPR = FP / (FP + TN)$ $MCC = (TP \cdot TN - FP \cdot FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$	Prove low false alarm generation and balanced classification under severe class imbalance (where attacks < 5% of flows)
Suite A: Threat Detection & Zero-Day Performance	**ROC-AUC, PR-AUC & Expected Calibration Error (ECE)**	Area under ROC & Precision-Recall curves; $ECE = \Sigma (\lvert B_m \rvert / N) \lvert acc(B_m) - conf(B_m) \rvert$ across 10 bins	Verify that the confidence score reported to SOC analysts reflects true empirical probability
Suite A: Threat Detection & Zero-Day Performance	**Unseen Zero-Day Holdout Recall (%)**	Detection rate on attack classes completely excluded from supervised training (e.g., Botnet / Heartbleed)	Prove that the unsupervised Autoencoder/PyOD branch catches novel attacks missed by supervised trees

Benchmark Suite	Metric Name	Mathematical Formulation / Definition	Research Purpose & Target Comparison
Suite B: Cross-Layer Correlation Ablation	**Alert Fatigue Reduction Ratio (AFR %)**	$AFR = (1 - N_{correlated_incidents} / N_{raw_alerts}) \times 100\%$	Quantify analyst workload reduction across *L1 Only* vs. *L1+L2* vs. *Full 3-Layer CyberVision*
Suite B: Cross-Layer Correlation Ablation	**Correlated False Alert Suppression Rate (FASR %)**	$FASR = ((FP_{raw} - FP_{correlated}) / FP_{raw}) \times 100\%$	Measure what percentage of single-layer false positives are suppressed by multi-layer graph correlation
Suite C: Explainable AI (XAI) Quantitative Metrics	**Explanation Faithfulness (Deletion Drop ΔP_k)**	$Faithfulness_k = (1/N) \Sigma [P(x_i) - P(x_i \mid Top\text{-}k)]$ when top-k (k=1,3,5) features are masked to benign median	Prove mathematically that SHAP/LIME identified the true causal features driving the model prediction
Suite C: Explainable AI (XAI) Quantitative Metrics	**Explanation Stability (Local Lipschitz $L(x)$)**	$L(x_i) = max \| \|\varphi(x_i) - \varphi(x'_i)\| \|_2 / \| \|x_i - x'_i\| \|_2$ under Gaussian noise $\varepsilon \sim N(0, 0.01\sigma)$	Benchmark stability of `TreeSHAP` vs. `KernelSHAP` vs. `LIME` (proving TreeSHAP is more consistent)
Suite C: Explainable AI (XAI) Quantitative Metrics	**Top-5 Jaccard Overlap & Latency ($ms/alert$)**	$J(A,B) = \|A \cap B\| / \|A \cup B\|$ between SHAP and LIME top-5 sets; wall-clock time per explanation	Demonstrate real-time feasibility of TreeSHAP (<15 ms/alert) compared to perturbation-based LIME
Suite D: Operational & Triage Efficiency	**Pipeline Throughput (EPS) & Mean Time to Triage (MTTT)**	Events processed per second (EPS); peer evaluation comparing Black-Box Alert vs. CyberVision XAI Card	Demonstrate practical SOC deployment speed and faster human decision-making

7. Engineering Guardrails Prevent Data Leakage and Reviewer Rejection

- **DO NOT Use Outdated KDD99 or NSL-KDD Datasets:** Reviewers at IEEE, Springer, and ACM conferences routinely reject papers relying on 1998 KDD Cup data. Stick strictly to CIC-IDS2017, UNSW-NB15, BETH, and CERT r4.2.
- **DO NOT Apply SMOTE or Scaling Before Train/Test Splitting:** Fitting `SMOTE` or `RobustScaler` on the entire dataset before splitting leaks test distribution statistics into training and invalidates your F1 benchmarks. Always split first, then transform training data only.
- **DO NOT Run KernelSHAP or LIME Over Millions of Benign Rows:** Compute XAI explanations selectively on flagged alerts (`P_hybrid >= 0.50`) and a stratified benchmark test sample of 500–1,000 alerts so your pipeline finishes in minutes rather than days.
- **DO NOT Query External APIs Inside Uncached Loops:** Deduplicate IP addresses and query AbuseIPDB, AlienVault OTX, and GreyNoise through a persistent SQLite cache.

8. Final Project Deliverables & Research Paper Handoff Protocol

Upon completing Phases 1–8 in your development environment, your team will have produced **six complete project deliverables**:

1. **Unified Multi-Layer ECS Database & Ingestion Pipeline** (`data/processed/cybervision_ecs.duckdb` + Pydantic normalizers).
2. **Trained & Calibrated Hybrid Model Suite** (serialized `XGBoost`, `LightGBM`, `RandomForest`, `PyOD`, and `PyTorch Autoencoder` models).
3. **Cross-Layer Graph Correlation & XAI Engine** (NetworkX multi-layer stitcher + SHAP/LIME explanation & narrative generator).
4. **Interactive CyberVision SOC Platform** (FastAPI REST service + 4-page Streamlit SOC Analyst Workbench).
5. **Reproducible Benchmark Tables & High-Resolution Plots** (`results/*.csv` + 300-DPI ROC/PR, confusion matrix, ablation, and SHAP plots).
6. **B.Tech Thesis Documentation & Conference Research Paper** (ready for Review 0–Final Viva and conference submission).

8.1 Research Outcomes Handoff Checklist (Send Back After Execution to Generate Your Final Paper)

Once your code execution finishes, share the following outputs in chat so your complete IEEE / Springer conference research paper can be generated with 100% factual precision:

- **Dataset Split Counts:** Exact train/validation/test row counts and class distributions used for Layer 1 (CIC-IDS2017 / UNSW-NB15), Layer 2 (BETH), and Layer 3 (CERT r4.2).
- **Suite A Classification & Zero-Day CSV Table:** Accuracy, Precision, Recall, Macro-F1, Weighted-F1, FPR, MCC, ROC-AUC, PR-AUC, ECE, and held-out zero-day attack recall across all baseline and hybrid models.
- **Suite B Multi-Layer Correlation Ablation Table:** Raw alert counts, correlated incident counts, Alert Fatigue Reduction Ratio (AFR %), and False Alert Suppression Rate (FASR %) across 1-layer, 2-layer, and 3-layer configurations.
- **Suite C Quantitative XAI Benchmark Table:** SHAP vs. LIME Faithfulness deletion drops (Top-1, Top-3, Top-5), Stability Lipschitz scores, Jaccard agreement %, latency (`ms/alert`), and top-5 most influential features per layer.
- **Suite D Operational Throughput & Target Venue:** Pipeline throughput (events/sec), mean end-to-end latency (ms), and your target conference format (e.g., IEEE 2-Column Conference Template, Springer LNCS, or ACM).